A Simulation and Evaluation of Distributed Algorithms

A Senior Project submitted to
The Division of Science, Mathematics, and Computing
of
Bard College

by
Ziyi Chen

Annandale-on-Hudson, New York
Dec, 2024

Abstract

Distributed systems consist of autonomous computing elements that operate collectively as a single coherent system. Achieving consistency across independent nodes in such systems poses significant challenges due to the absence of a central clock, which complicates event ordering and state coherence. Logical clocks, such as Lamport's Logical Clock algorithm, address these challenges by enabling event ordering and causal dependency.

This paper explores methods to ensure consistency and proposes approaches to measure a simulated distributed storage systems. The system, implemented in Golang, examines consistency under various conditions, employing defined inconsistency metrics to evaluate read/write performance. The study provides insights into the mechanisms of logical clocks and their effectiveness in maintaining consistency.

Contents

Abstract iii

Acknowledgments vii

1 Introduction 1

2 Background 3
- 2.0.1 The Raft Algorithm . . . 4
- 2.0.2 Distributed Storage . . . 5
- 2.0.3 Synchronization . . . 6
- 2.0.4 Consistency . . . 6

2.1 Research Problems . . . 7

3 Methodology 9

3.1 Introduction . . . 9

3.2 Simulation Design . . . 9
- 3.2.1 Node Model . . . 9
- 3.2.2 Event & Version Chain . . . 11
- 3.2.3 Exchange/Broker . . . 11
- 3.2.4 Redis . . . 12

3.3 Clock Synchronization . . . 13
- 3.3.1 Tie Breaking . . . 16
- 3.3.2 Causal Dependency . . . 16

4 Simulation Analysis 19

4.1 Consistency Definitions . . . 19

4.2 Overview of Simulation Experiments . . . 20

4.2.1 Simulated Storage Systems . . . 20
4.2.2 Operation . . . 21
4.2.3 Client-Side vs Server-Side . . . 21
4.2.4 Simulated Data and Version Chain . . . 22
4.2.5 Simulation Execution . . . 22
4.2.6 Consistency Analysis . . . 23
4.3 Consistency Metrics . . . 24
4.3.1 Client-Centric Metrics . . . 24
4.4 Results . . . 27

5 Discussion **31**
5.1 Results Analysis . . . 32
5.1.1 Read/Write Proportion . . . 32
5.1.2 Latency . . . 32
5.1.3 Scalability . . . 33
5.1.4 Message Loss . . . 33
5.1.5 Event Frequency . . . 34
5.2 Conclusion . . . 34
5.3 Future Work . . . 35

Appendices **37**

A Source Code **37**

Bibliography **39**

Acknowledgments

I would like to thank my advisor, Sven Anderson, for his guidance and insight throughout my senior project and academic journey at Bard College.

I am thankful to Bard College for awarding me a generous scholarship, which not only enabled my academic pursuits but also provided me the opportunity to immerse myself in the unique intellectual and cultural life of the United States, the reason I chose to study abroad.

To Lily Lin, thank you for your love, companionship, and support over these challenging four years of college life.

To my family, who have always been my pillar, supporting my education and shielding me from misfortune.

Lastly, my appreciation goes to the Computer Science Program at Bard College. They have challenged my perspectives on computer science and the software engineering industry, inspiring me to grow intellectually.

1
Introduction

A distributed system is a collection of autonomous computing elements that appear to its users as a single coherent system[1]. In distributed systems, achieving consistency between independent nodes presents a significant challenge. These systems lack a central clock, making it difficult to determine the order of events and maintain a coherent state. Logical clocks, such as Lamport's Logical Clock algorithm, have been proposed as solutions to address this problem by enabling event ordering without requiring physical clock synchronization.

This paper revolves around algorithms, like Lamport's Logical Clock Algorithm, that enable distributed systems' data storage to remain consistent in a certain agreed-upon manner. How is this consistency reliably ensured? Furthermore, what approaches there are to measure the consistency of distributed storage systems?

This study contributes to the understanding of logical clocks by implementing a simulated system that is examined under a variety of conditions. To explore notions of consistency and how much it can be guaranteed, inconsistency metrics are defined and then applied to the simulation output.

2
Background

In modern software engineering, distributed systems have become a foundation of various industrial applications. Traditional centralized (monolithic) systems can no longer meet the growing demands of scalability and reliability. The definition of distributed systems varies, but a common characteristic is that they operate across multiple nodes (computers).

It is evident that distributed systems are now more commonly integrated as a feature of specific services rather than existing as standalone systems. For instance, both *PostgreSQL* and *Redis* implement modules natively cache hotspot data in memory in the form of pages, and evict pages to disk when the memory cache is full. This process of transferring data from memory to disk is also referred to as *swapping*. If a system that implements swapping is categorized as a "Caching System," then does this imply that PostgreSQL and Redis should be called "Caching Systems"? Clearly, that is not the case. PostgreSQL is still classified as a relational database, while Redis is recognized as an in-memory key-value storage service or, more broadly, a *NoSQL* database.

Distributed systems have always fascinated me, as my experience developing and maintaining them grows. When reading the official documents, tech blogs, and source code, *Raft*, an distributed consensus algorihthm, drew my attention. This algorithm is behind many famous services and middleware, e.g.,

- **Redis** k-v in-memory storage. Can be cluster-deployed.

- **etcd** a system specifically designed to coordinate distributed data consistency.

The well-known online free course *MIT 6.824: Distributed Systems* mainly focuses on the implementation of **Raft**. Distributed consensus algorithm means that it handles the consensus, a shared information among nodes within the system. Inspired by the course, Raft sets up an outline about the implementation of the simulation system the following chapter describes.

2.0.1 The Raft Algorithm

As a background, the *Raft* algorithm was based on **Paxos**, an even earlier distributed consensus algorithm, which grew from the work by Leslie Lamport, where the problem of sharing data among nodes within a distributed system was first described. “Assume a collection of processes that can propose values. A consensus algorithm ensures that a single one among the proposed values is chosen...The safety requirements for consensus are...Only a single value is chosen. Assume that agents can communicate with one another by sending messages. We use the customary asynchronous, non-Byzantine model, in which Agents operate at arbitrary speed, may fail by stopping, and may restart. Since all agents may fail after a value is chosen and then restart. A solution is impossible unless some information can be remembered by an agent that has failed and restarted[2].” To rephrase Lamport’s idea on consensus of a distributed system, it concludes the distributed system as whole responds to, for example, external read operations, as a whole, returns a value (data). This value is formed based on a consensus from the ”agents”, the *nodes* composing the distributed system. And this value, necessarily a single value, which means only one chosen value all the nodes agree upon being returned, is defined as the safety of consistency.

However, it is likely to have a distributed system return multiple divergent values returned by read operations at specific time. Paxos states that nodes must be able to communicate with one another by sending messages and storing states. In addition, “Messages can take arbitrarily

long to be delivered, can be duplicated, and can be lost, but they are not corrupted[2]." The same rules will apply to the simulation system.

2.0.2 Distributed Storage

Distributed storage refers to a data storage architecture in which data is stored across multiple physical locations, such as servers or data centers, rather than being confined to a single device or server. Notable examples of distributed storage systems include distributed file systems like Hadoop Distributed File System (HDFS). In some systems, data are **duplicated** across nodes, which means every single node is expected to hold a complete copy of the entire data. In this case, *node* and *replica* are equivalent.

Node Distributed system is a collection of computing elements each being able to behave independently of each other. A computing element, which we will generally refer to as a **node**, can be either a hardware device or a software process. A fundamental principle is that nodes can act independently from each other[1]. In practice, nodes are programmed to achieve common goals

Local storage refers to the storage resources that are directly attached to a single node from the distributed system. It provides quick data access and is essential for processing tasks locally before any data is shared or synchronized with the rest of the system. The use of local storage can be crucial for optimizing performance in tasks that require low-latency access to data, since some computation can be done once locally to avoid redundant computation on multiple nodes. The Amazon Elastic Compute Cloud, or simply EC2, allows one to create an environment consisting of several networked virtual servers, thus jointly forming the basis of a distributed system. Virtual servers load AMIs (Amazon Machine Images), installable software packages consisting of an operating-system kernel along with a number of services. For example, a node has a Linux kernel, the Apache Web server, a MySQL database system[1] installed, and the local MySQL database instance is namely *local storage*.

2.0.3 Synchronization

When dealing with data synchronization, the problem is to ensure that two sets of data are the same[1]. A way to understand data as arising from a series of operations, and therefore the order matters. The order determines the current value of the data. Hardware clocks, and thus also software clocks, are subject to clock drift: because their frequency is not perfect and affected by external sources such as temperature, clocks on different machines will gradually start showing different values for time[1]. Due to the randomness of real-world latency, there is no absolute approach, if nodes of a system are merely connected through network, to secure a **global clock**. Nodes do not have a common time reference, and each one will have its own notion of time[1].

Therefore, the **Logical Clock** is introduced. Denote vid_n as the data stored in the system. For running make it is adequate that two nodes agree that vid_1 is outdated by a new version of vid_2, for example. In this case, keeping track of each other's events (such as producing a new version of vid_2) is what matters. For these algorithms, it is conventional to speak of the clocks as logical clocks[1]. A logical clock is applied in the simulation system to synchronize data.

2.0.4 Consistency

Consistency is a complex concept. There is strict consistency level, **Linearization**, which not only requires data to be identical in terms of content and order, also states that there must exist a known time interval, in which data is written to every nodes. All the writes must start and successfully finish during that known interval. This is not quite possible for the simulation system, given that no global clock as the shared reference of time is available. Let's say the system checks data at all nodes at a possibly maximum frequency, and ensuring that data and order are synchronized correctly, the system might be able to be claimed close to Linearization-level consistency. However, **CAP theorem** states that **consistency**, availability, and partition tolerance cannot be simultaneously achieved in a distributed system[7]. If Linearization-level consistency is aimed at, the system's availability becomes challenging. Weaker consistency models must be taken into consideration. From weak to strong there are WC (Weak Consistency),

EC (Eventual Consistency), CC (Causal Consistency), and Linearization. Later in the paper these models will be further discussed. CC is the strictest level of consistency that can be achieved in an always available storage system based on the trade-offs of the CAP theorem. In a causally consistent storage system, all requests that have a causal relationship to another request must be serialized (i.e., executed) in the same order on all replicas. A request r_2 causally depends on a request r_1[7]. These requests, for the simulation system as a storage system, are **Read** and **Write** operations (also called events, will discussed soon). A typical implementation uses vector clocks to identify (potential) causal dependencies[7]. In the simulation, Lamport's Clock algorithm is implemented. Clock algorithm stipulates a set of rules how Logical Clocks are generated and compared.

2.1 Research Problems

Research covered in this paper mainly focuses two aspects:

(1) A correct implementation of simulated distributed systems.

(2) An analysis of system consistency executing read/write under a set of experiment conditions.

More specifically, the first issue resolves around an executable computer program that simulates an distributed system and leverages certain existed algorithm to facilitate data consistency. In the Methodology chapter, the implementation detail is discussed.

How is Lamport's Timestamp's effectiveness at maintaining consistency in distributed systems under conditions like delay and message loss? This project proposes consistency analysis and metrics to measure the simulation system.

3
Methodology

3.1 Introduction

There exist various models for distributed systems depending on the features we allow: message passing model, shared memory model, mobile robots model, etc. Here, we restrict our study to the standard message passing model for distributed storage. The communication consists of a point-to-point communication network described by a connected graph G, where the nodes represent network processes and the edges represent bidirectional communication channels. Processes communicate by message passing: a process sends a message to another process by depositing the message in the corresponding channel. We assume the system is asynchronous, namely, all processes act independently and there is no global clock[4].

3.2 Simulation Design

3.2.1 Node Model

The proposed distributed storage system comprises a series of independent, autonomous, isolated runtime (consider the Amazon EC2 virtual machine installed with an operating system kernel and a database). The simulation system is fully developed in *Golang*, a language that specializes at concurrency. Each node is implemented inside a goroutine. To convenient purpose, within

this paper, **node** is used to refer to the code being run in a goroutine. The code is wrapped by `Machine interface`.

```
type Machine[T Payload, K ClockDataType] interface {
    Start()
    Stop()
    Init()

    publish(VersionPtr[T, K], exchange.Exchange)
    poll() *option.Option[VersionPtr[T, K]]
    Listen()
    Broadcast(VersionPtr[T, K])

    BindSub(recv exchange.Exchange)
    BindPub(send exchange.Exchange) error

    LocalEvent(event func(data T) T)

    GetId() uint64
    GetVersionChainData() []T
}
```

No Master Node The system is a peer-to-peer (p2p) system. All nodes are identical (symmetric). There does not exist a coordinator e.g. a master, write-exclusive, or sentinel node that is responsible for something more than any other node. In *Raft*, there is election algorithm that produces leader node, while this simulation assumes a p2p system.

Golang's Coroutine

Coroutine is a control structure widely used in computer programs. It is a relative new thing, as some programming languages have not supported it as a feature. Python's async/await keywords that mark coroutines were introduced in Python 3.5. Coroutine is designed for multitasking. It is less memory-consuming and brings less switch overhead. It is similar to thread, but originally it is non-preemptive, which means that A_{coro} has to yield so other B_{coro} can run. Otherwise, A_{coro} will occupy the computing resource until it ends.

Goroutines, inspired by coroutines, are preemptive, more efficient than both thread and coroutine. Goroutines are managed by Go's runtime, and are friendly to mass duplicate and to programmed. An experiment condition requires 200 nodes to run. Generally speaking, having 200 goroutines is much easier than having 200 threads.

3.2.2 Event & Version Chain

Through `Machine interface`, read/write operations can be invoked externally. Following local events directly modify local storage:

Read `GetVersionChainData() []T` returns a list of history data of `type T`.

Read or Write `LocalEvent(event func(data T) T)` requires a callback parameter as the procedure that will execute on the data.

Each node maintains a version chain, perform **local read/write** operations on the stored data by modifying the version chain. `VerChain *utils.LinkList[Version[T, K]]` is a pointer to a linked list where the history data are stored. The history data, in this simulation system, is the *local storage* discussed earlier. The version chain indicating orders and contents of history operations/events, might be inconsistent with these of other nodes. From the perspective of a single node, the state of the entire distributed system is always unknown.

3.2.3 Exchange/Broker

`Exchange interface` offers function stabs to transmit messages across nodes. Nodes are bound to `Exchange` instances, `Broker`, so they can publish/subscribe to that `Exchange`. `Broker` simulates randomized delay and packet loss and is configurable by the external caller. The implementation was inspired by the famous Python messaging library *celery/kombu*.

```
type Exchange interface {
    Put(t []byte) bool
    Get() []byte
    C() <-chan []byte
    GetId() uint64
}
```

Notice that messages are serialized, or "marshaled", before sent to the `Exchange`. They are converted to byte array.

Channel

Goroutine, combined with Go's built-in **channel**, provide a structured and safer approach to concurrent programming. Channels reduce programmer's cognitive load by coordinating senders

and receivers: if there's no available value while a receiver is getting from it, the receiver gets blocked until some sender finally puts a value into it. Programmers write Go channel code and put and get are natively thread-safe.

More than this, channel makes implementing the *Proactor Pattern* easy. Redis is implemented in this style. When a socket is blocked, the program will not block to wait for it, as goroutine already enables that. But if the control flow will have to poll the socket everytime so it knows when the socket is ready, it is not proactive. In Proactor Pattern, the socket will notify the program, or the scheduler, so the scheduler does not have to poll at all.

3.2.4 Redis

Mainly inspired by Redis, the development of the system refers to the source code of Redis a lot. Redis is a distributed, in-memory (it keeps data in memory, not hard dist) key–value database or cache. Program written in any programming language with a built Redis interface/library, for example, *redispy* for Python, is able to access to the Redis service. Redis service is of its own process, a background task started by Linux user.

To integrate Redis in a Python program, for instance, call its API to create a stub. The stub holds a connection to the running Redis process behind the scene. Two common operations are **get** and **set**, similar to Python dictionary.

- Get/Read - retrieve certain data from the service.
- Set/Write - add or overwrite certain data on the service.

The simulation system developed for this paper is a similar data storage system. Local reads and writes happen on the node (as `r` in the code below, the `Machine` in the simulation system). From external perspective, as in Redis, any number of operations can be executed in parallelism style, rather than single-threaded, serialized requests. In another word, both Get and Set do not block the external control flow. It is done instantly while a messaging algorithm synchronizes data, so the Get/Read guarantee consistent result, which means the same value will be retrieved

on any connection to one Redis service in any program. This Eventual Consistency is very vague in terms of concrete guarantees but is very popular for web-based services[9].

```
import redis
r = redis.Redis(host=redis_host, port=redis_port)
r.set('name', 'Alice')
r.set('age', 30)
name = r.get('name')
age = r.get('age')
```

3.3 Clock Synchronization

Leslie Lamport proposed a timestamp-based algorithm for **logical clock** synchronization. To give an overview, each node maintains a monotonically increasing integer as timestamp. The larger the timestamp the newer the its state is[5].

Nodes coordinate by comparing timestamps. Timestamp should never decrease because decrease means reverting to an old version.

3.3.0 Procedure

- **Step 1** Local write event (string: vid_a) takes place on $Node_A$. Store the data and increment the timestamp to ts_A.
- **Step 2** $Node_A$ broadcasts message M_A containing (1) the value of ts_A (2) string: vid_a (3) Node id of the $Node_A$ to other nodes within the system.
- **Step 3** $Node_B$ receives M_A. Compare its timestamp ts_A with local timestamp ts_B. If $ts_A > ts_B$, the data vid_a and ts_A overwrites $Node_B$'s local storage. If $ts_A < ts_B$, M_A is aborted.
- **Step 4** Increment $Node_B$'s timestamp by 1.

Figure 3.3.1 demonstrates that without message broadcasting, no data synchronization, and no consistency guaranteed. Three machines result in data B, C, D. Figure 3.3.2 demonstrates a naive total broadcasting results in consistency that is very weak. The eventual data completely

depends on a random order of message sending and receiving, due to the unknown travel time of each messaging. Weak consistency translates to a colloquial “replicas might by chance become consistent”. Weak Consistency does not provide any ordering guarantees at all[9]. Three machines result in data D, A, C.

In Figure 4.2.3, Lamport’s Timestamp algorithm successfully guarantees Eventual Consistency. It requires convergence of replicas, i.e., in the absence of updates and failures the system converges towards a consistent state. Updates may be reordered in any way possible and a consistent state is simply defined as all replicas being identical[9]. Nodes compare timestamps to decide how new the message is and then to take new data overwriting the old. All 3 nodes agree upon the same data D as the final state, marked by yellow boxes.

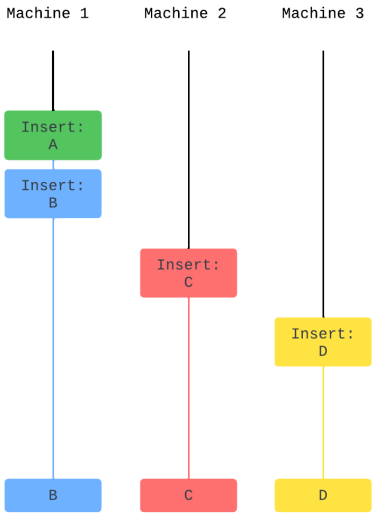


Figure 3.3.1: No Synchronization

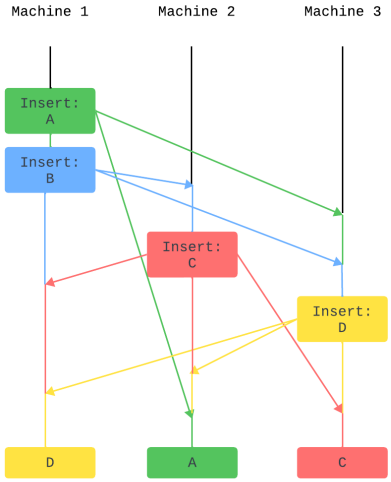


Figure 3.3.2: Naive Data Broadcasting

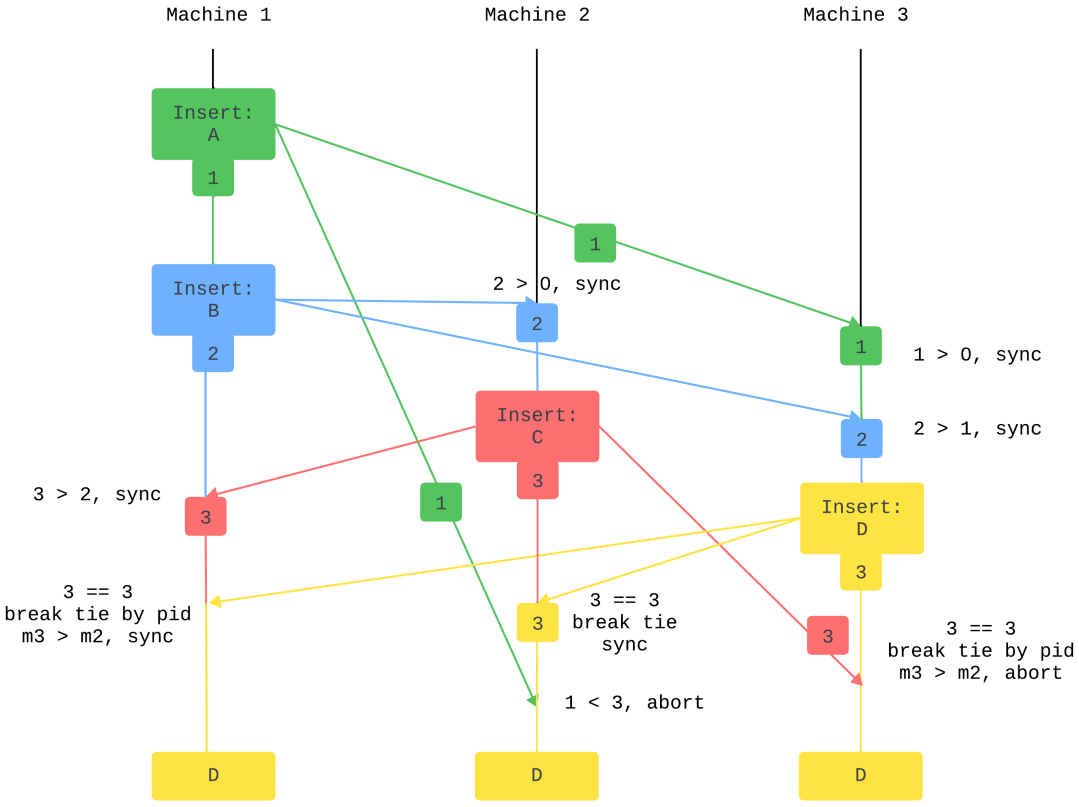


Figure 3.3.3: Lamport's Timestamp

3.3.1 Tie Breaking

It is possible for two nodes to have near-simultaneous write operations and then exchange synchronization messages, carrying an equal value of timestamp. Moreover, a node increments timestamp first, broadcasts later. So $ts_A = ts_B = ts_{Node_A} = ts_{Node_B}$ holds.

To break the tie, when timestamps are equal, the node with the larger Node Id takes precedence. Notice that Node Id is a positive integers uniquely allocated to a node. Generally, clocks that define causal relationships can be in conflict (e.g., for unrelated concurrent updates). If clocks are used for request ordering and an approach exists that defines a transitive, global order for all conflicting clocks[9], then the consistency is still guaranteed. Node Id is exactly "the approach that defines a transitive, global order for all conflicting clocks". This is because when clocks conflict, another word, tie, happens, each node does not need to be offered total ordering, but still know that there is a commonly guaranteed order: there **is and only is** one greatest Node Id among any number of nodes that are confused by conflicted messages.

3.3.2 Causal Dependency

The Version Chain is, in fact, a linked list, where each listnode carries a past version: data string, old timestamp, and Node Id of the data's source node.

To synchronize logical clocks, Lamport defined a relation called happens-before. The expression $a \rightarrow b$ is read "event a happens before event b"[5]. In this paper it means that one node agrees that first event a occurs, then afterward, event b occurs[5].

Version chain always append newest data at the end of the linked list. Write out the data from the linked list and connect with arrow, denoted as: Vid1 $\rightarrow$ Vid2 $\rightarrow$ Vid3. How does version chain guarantees that the happen-before relations it describes are meaningful, even accurate? Recall the **3.3.0 Procedure**. Modify the last step as follow:

Step 3 $Node_B$ receives M_A, then insert ts_A into $Node_B$'s *version chain*, the linked list, at where it finds the listnode contains ts_x that is not newer than ts_A, where $ts_x \leq ts_A$ holds.

It is easy to see from the definition of the relation $\rightarrow$ that the Clock Condition is satisfied if the following two conditions hold[5].

- **C1**. If a and b are events in node P, and a comes before b, then a $\rightarrow$ b.
- **C2**. If a is the sending event of a message by node P_i and b is the receipt of that message by node P_j, then a $\rightarrow$ b.

[5]Since causality is transitive, in the end all events (local write, local read, send, recv) will fall into three categories: cause(past), effect(future), independent(concurrent). Therefore, a total causal dependency is established. A complete causal dependencies tell about the causation relations among all the known events within a distributed system. "We know what we know, and we don't know what we don't know" is the way how Causal Consistency is defined. Lamport's Timestamp algorithm only guarantees the consistency that a node's dependencies information lead to. As long as the system is ideal, CC is guaranteed, as the next chapter illustrates. However, when the external module generates randomized sequence of operations to invoke the system, extra information are available. The information, as discussed in the next chapter, conclude the ordering, frequency, and latency of operations. Causal dependencies, equivalently the version chains, could contradict with these extra information. The nodes are expected to be able to guarantee Eventual Consistency at least as the original goal of the implementation, which can not be formally verified yet. The consistency of the system under experimental conditions requires further verification.

```
1  func (u *LamportClock) DefaultTsCmp(v DistributedClock[int]) TsOrder {
2      if u.counter > v.Value() {
3          return BEF
4      } else if u.counter == v.Value() {
5          return CON
6      } else {
7          return AFT
8      }
9  }
10
11 func (l *LinkList[T]) InsertBefore(data *T, cmp LinkListCmp[*T]) {
12     prev := l.Head
13     runner := l.Head.Next
14
```

```
15      for runner != nil {
16          if cmp(runner.Data, data) {
17              tmp := NewNode[T](data)
18              prev.Next = tmp
19              tmp.Next = runner
20              return
21          }
22          prev = runner
23          runner = runner.Next
24      }
25      prev.Next = NewNode[T](data)
26  }
```

4
Simulation Analysis

This project evaluates distributed systems using a variety of consistency metrics. These metrics are explained and then applied to the simulated systems.

4.1 Consistency Definitions

When discussing **consistency** of distributed systems, further definition of the consistency model is required. In 1979, Leslie Lamport provided a strict definition known as **Sequential Consistency**: *"The result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program"*. By contrast **Eventual Consistency** primarily focuses on the execution's outcome. For instance, consider the following Write Event Log A: Vid1 $\rightarrow$ Vid2 $\rightarrow$ Vid3 (where $\rightarrow$ denotes "precedes"). Since each write operation overwrites the previous data, the final result of Log A will be equivalent to Log B: Vid1 $\rightarrow$ Vid1 $\rightarrow$ Vid3: that is, both end in state Vid3. These two node logs do not violate Eventual Consistency. However, Lamport's model also constrains the order of execution. Under this stricter rule, logs A and B would violate **Sequential Consistency**.

Eric Brewer's well-known **CAP theorem** extends Lamport's Sequential Consistency to propose a more stringent model called **Atomic Consistency**, also known as **Linearizability**. Linearizability is considered the strictest type of consistency. It assumes the existence of a

global clock and requires not only that the local operations within each process are sequential but also that operations occurring simultaneously *"take effect at a point within the time interval"* [8]. Our model, however, is based on the assumption that a global clock does not exist. While the **version chain** can determine the order of operations on any given node, it cannot establish whether events occurred simultaneously across nodes; consequently, we do not evaluate system linearizability.

The CAP theorem states that **consistency**, **availability**, and **partition tolerance** cannot be simultaneously achieved in a distributed system. All three are desirable for real-world web services [7]. As a result, consistency must often be weakened to ensure the other two properties. Gilbert and Lynch discuss various definitions of **Weak Consistency**, which describe the conditions under which clients expect operations to maintain a specific order and outcome. Some weak consistency models are now used as benchmarks for distributed storage systems, becoming widely applied in the industry. For instance, as discussed in Section 3.2.4, Redis can be measured using the Eventual Consistency (EC) model. If Redis satisfies the EC model, it is called an *Eventually Consistent Storage System*[7].

Bermbach and Kuhlenkamp further explored methods for measuring weak consistency models. They state: *"A system is in a consistent state if all replicas are identical and the ordering guarantees of the specific consistency model are not violated"* [9]. Based on their definition, this paper first defines violations of Weak Consistency Models, then runs simulations to record the frequency of such violations. We aim to quantify how consistent the distributed system is under specific conditions and determine how these conditions affect consistency.

4.2 Overview of Simulation Experiments

4.2.1 Simulated Storage Systems

The systems that are simulated are distributed storage systems, comprised of interconnected Nodes. Each node corresponds to an isolated runtime environment in the real world, equivalent at least to a virtual machine. In this paper, **Nodes** are also referred to as *Machines* or *Replicas*.

The collection of nodes is called a *cluster*, and the **Cluster Size** is the number of nodes in the system. Every two nodes are connected by a communication line that enables data transmission, referred to as *Send/Recv of Sync Message/Messaging*.

4.2.2 Operation

The *Operation*s are procedures executed on the data. Operations are either *Read* or *Write*: Read operations passively view the data, whereas write operations modify the data. The simulation module predefines a series of Reads and Writes randomly distributed over the time of the simulation run. A sequence of instructions executed by a node is an *event*. Both Read and Write are considered *local events* because they directly affect the local storage. **Data-centric** domain namely encapsulates the operations of direct data modification, so that the external system is disable to directly modify the data storage.

4.2.3 Client-Side vs Server-Side

Bermbach and Kuhlenkamp distinguish **Data-centric** and **Client-centric**, referring to Server and Client domains. In the system implemented in this paper, a client can be represented by the interface *Machine*, which provides Write and Read function stubs to simulation modules that are external to the machine. The client side views the system as a black box.[9] Notice that nodes cannot initiate operations on their own; they execute operations as results of Client calls. All reads and writes occur only locally, meaning that for a system with n Nodes, there are n Clients. The reader can imagine a *Shared Document application* running on two computers. Any change made on the document is first executed locally and then synchronized to the "cloud storage", then propagated to other nodes. However, operations are not directly executed on the "cloud storage"; they must go through the clients.

4.2.4 Simulated Data and Version Chain

The specific data stored on nodes and is a short message string. The most recently written string becomes the latest data and is appended to the Version Chain. The Version Chain is, in fact, a linked list, where each *listnode* carries a past version: data string, old timestamp, and Node Id of the data's source.

The simulation module manages the data, a 1-indexed monotonically increasing, consecutive integer, referred to as **Version Id**. Version Id only increments externally when a Write operation is invoked, not directly by Nodes. Notice that the order of Version Id stored on a Node might not be monotonically increasing, which indicates that an inconsistency happened.

As a convenience the term **version chain** is used to refer to a list that contains all Version Ids from the Linked List, denoted as: Vid1 $\rightarrow$ Vid2 $\rightarrow$ Vid3. Version chain here is equivalent to **Operation Log/Event Log** used in other works.

4.2.5 Simulation Execution

Operations and Events are discrete. The system assumes that Operations are never lost, and there is no known Latency or Fault in messaging. Consequently, the total number, type (read-/write), and order of Operations should be identical to those of Local Events. Operations are continuously invoked in a loop and counted. A positive 1-based integer represents the iteration, such that: *Iter (Iteration) = Epoch = (Local) Event Count = Operation Count.* For an experiment, the Event Count is fixed. For example, an experiment only examines the consistency metrics of the system at the end of 10,000 Read/Write operations. We assume that events occur randomly at a fixed rate via a Poisson process, yielding exponentially distributed event interarrival times[11]. The system implements a Poisson random process to determine the frequency of operation events by adjusting the **Average Event Interval**.

Goroutines are *stackful coroutines*, meaning each goroutine maintains its own stack, which is managed in heap memory. Very high frequency of operations might bring an extra amount of memory allocation/free. Golang's GMP model says that if there are more goroutines than

kernel threads, multiple goroutines are attached to one kernel thread and take turns to run. Each event is likely to trigger a switch and reschedule, especially when there are more Nodes than the CPU cores on the physical machine. When the time interval of events is close to the time length of the kernel thread's time slice, indicated by `sysctl_sched_min_granularity` and `sysctl_sched_latency` parameters in Ubuntu, typically in the range of 10^{-1} to 10^{1} seconds, it is more likely for the operating system to schedule the kernel thread and goroutine attached to that thread together, reducing context switch overhead. Therefore, the **Average Event Interval** is set at 25 ms, 60 ms, or 180 ms.

Similarly, messaging between Nodes follows a Poisson random process. In the best case, two servers on opposite sides of the Earth—communicating via a hypothetical link through the planet's core—require a minimum round-trip time (RTT) of 85.1 ms (133.7 ms if sent at the surface level)[10]. The simulated messaging latency is set at 40 ms or 180 ms.

A 2010 study of over 200 wide-area routers found an average of 16.2–302.0 failures per link per year, with an average annual downtime of 24–497 minutes per link (95th percentile: at least 34 hours)[10]. Based on this data, the ratio of total down time (time when the network is unavailable) and the total time of a year, is roughly considered a baseline **Packet Loss Rate** (PLR) of 0.388%. To contrast, high PLR is set at 1%.

4.2.6 Consistency Analysis

In the synchronization algorithm of the simulation system, a local write event is broadcast to all Nodes. Assuming zero PLR and Latency, the Version Chain is expected to remain identical across all Nodes, because Timestamp offers enough information, partial orders, or causality, accurate and sufficient, suggesting at least *Causality Consistency* is guaranteed. This is a rather strict Consistency model. If stronger consistency is guaranteed, weaker consistency, such as Eventual Consistency (EC), can also be expected. In a 4-Node simulation with 0 PLR, 0 Latency, a sample of 16 runs at 10,000 event count gives following results (95% CI):

$R_{MRV} = 1.9[1.5, 3.8], \quad R_{RYWV} = 0.7[0.4, 1.1], \quad R_{WFRV} = 1.1[0.6, 1.6], \quad R_{t\text{-}visibility} = 8.3[4.4, 12.1]$

These results suggest that the implementation of a CC system cross validates with the Consistency Metrics that the following section will discuss. Under favorable conditions, the system's performance tends to yield metrics close to 0. The consistency is close to at least EC.

4.3 Consistency Metrics

Both data-centric and client-centric consistency guarantees have two dimensions: ordering and staleness. Staleness describes how much a given replica lags behind, expressed in terms of time (t-visibility)[9].

- ***t*-visibility** refers to the time difference in data updates across different replicas, representing the elapsed time from when one replica starts updating to when another replica finishes updating. In the simulation program, t-Visibility is the time elapsed from *when the write started* to *when all nodes complete writing and the data becomes available through read*. This Data-centric metric can only be measured from inside of the system.

$$t_{\text{Visibility}} = \max\left(T_r - T_w\right)$$

4.3.1 Client-Centric Metrics

By analyzing ordering and staleness, an approach to quantify consistency metrics is available. We mine the replicas' operation logs to determine the number of violations for each of the consistency models[9]. By recording how many times a violation of a certain following model is recorded, the performance of the system in terms of data synchronization is measured. There are four client-centric metrics considered in our analysis.

- **Monotonic Read Consistency** (MRC), guarantees that a client that has read a version n will thereafter always read versions $\geq n$[9]. An MRC violation occurs when last Read gives $vid_{last} = n$ (Version Id), and current Read give $vid_{cur} < n$.

- **Read Your Writes Consistency** (RYWC) guarantees that a client that has written a version n will thereafter always be able to read a version that is at least as new as n, i.e., $\geq n$ [9]. Consider the following example in the Lamport's Clock system:

  - $Node_A$ has missed many previous versions, with its current $ts_A = 2$ (Timestamp).
  - $Node_B$ has the most recent version so far, vid_{n-1}. with $ts_B = 5$, but it's broadcasting a Message carrying $M_B(5)$ with vid_{n-1} to A carrying the Timestamp 5, and the Message is **delayed**.
  - Local Write happens on $Node_A$, recorded as $ts_A = 3$. $Node_A$'s local data is noted vid_n.
  - Then $M_B(5)$ arrives at $Node_A$. Since $ts_B(5) > ts_A(3)$, vid_{n-1} overwrites $Node_A$'s data vid_n.

  Now a violation to RYWC can be observed, as $Node_A$ that has written a version vid_n will read $vid_{n-1} < vid_n$, which contradicts RYWC's definition.

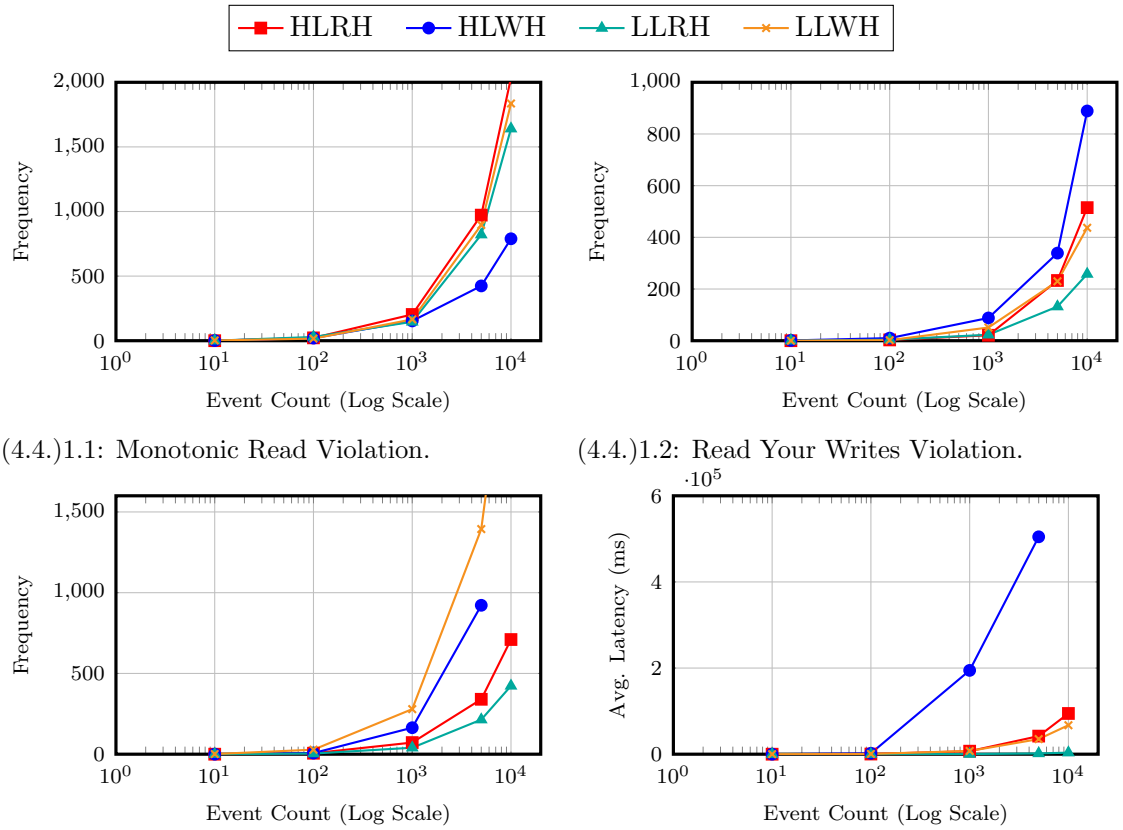
- **Monotonic Writes Consistency** (MWC) guarantees that two updates by the same client will be serialized in the order that they arrive at the storage system[9]. The Version Ids, as discussed above, are generated from a node-external module. Any Node within the system is expected to have the Version Chain in exactly the same order, monotonically increasing. An MWC violation is defined as a fault to this order, e.g. Vid2 $\rightarrow$ Vid1 (reversed order), Vid3 $\rightarrow$ Vid3 (repeated vid), Vid32 $\rightarrow$ Vid34 (non-consecutive). This metric differs from MRC as it indicates the cumulative faults in the entire past Version Chain, while MRC only indicates that a recent Read shows inconsistency during a certain time window.

- **Write Follows Read Consistency** (WFRC) guarantees that an update following a read of version n will only execute on replicas that are at least of version n. So far, no benchmarking approach exists for WFRC violations[9]. Similar to the case described in

MYWC, the update vid_{n-1} takes over vid_n happens, while $n-1<n$. No metric for WFRC is proposed here, since it can be measured through WFRC metric.

4.4 Results

This section contains three figures (Figure 4.4.1, 4.4.2, 4.4.3). Each figure consists of 4 subplots, each showing a consistency metric from MRC violation, EYWC violation, MWC violation, and $t_{visibility}$. Experiment conditions are marked by colors, with legends and experiment parameters noted around the figure. One subplot shows the value of the consistency metric across conditions.



(4.4.)1.1: Monotonic Read Violation.

(4.4.)1.2: Read Your Writes Violation.

(4.4.)1.3: Monotonic Write Violation.

(4.4.)1.4: t-visibility.

Figure 4.4.1

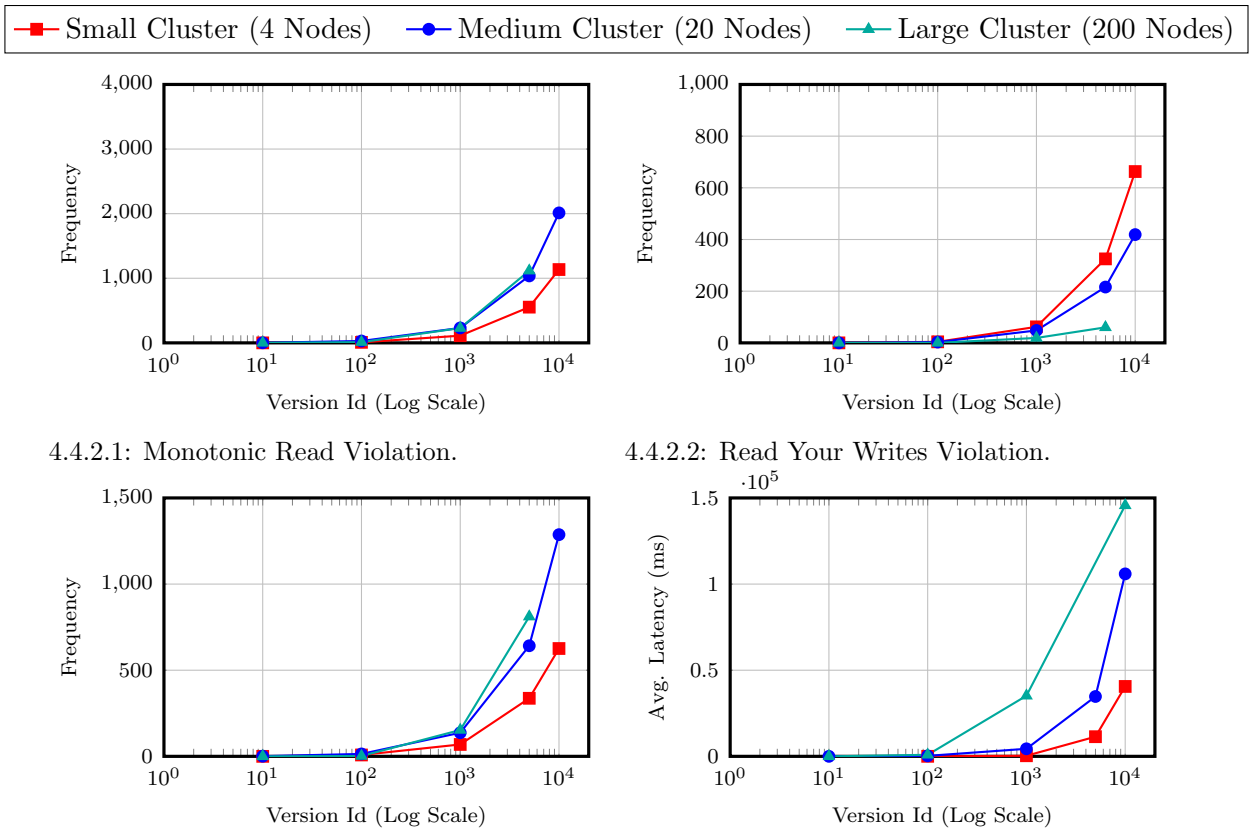
	Avg. Latency (ms)	Read Prop.	Node
High Latency Read Heavy	180	0.8	4
High Latency Write Heavy	180	0.2	4
Low Latency Read Heavy	40	0.8	4
Low Latency Write Heavy	40	0.2	4

Each subplot in Figure 4.4.1 shows performance as measured by one Metric. The table shows other parameters used in these experiments. In 3 of the 4 subplots, the green line (LLRH)

remains at the bottom, exhibiting the slowest growth rate as the event count increases. For these three Metrics, LLRH performs the best, indicating that the LLRH condition enhances system consistency.

The blue line shows the highest inconsistency in both RYWC Violation and t-visibility Metrics. Additionally, it has the second-highest cumulative occurrences in MWC Violation, suggesting that HLWH results in poor system consistency.

Unexpectedly, the orange line (LLWH) presents has frequent violations, further discussed in next chapter.



4.4.2.1: Monotonic Read Violation.

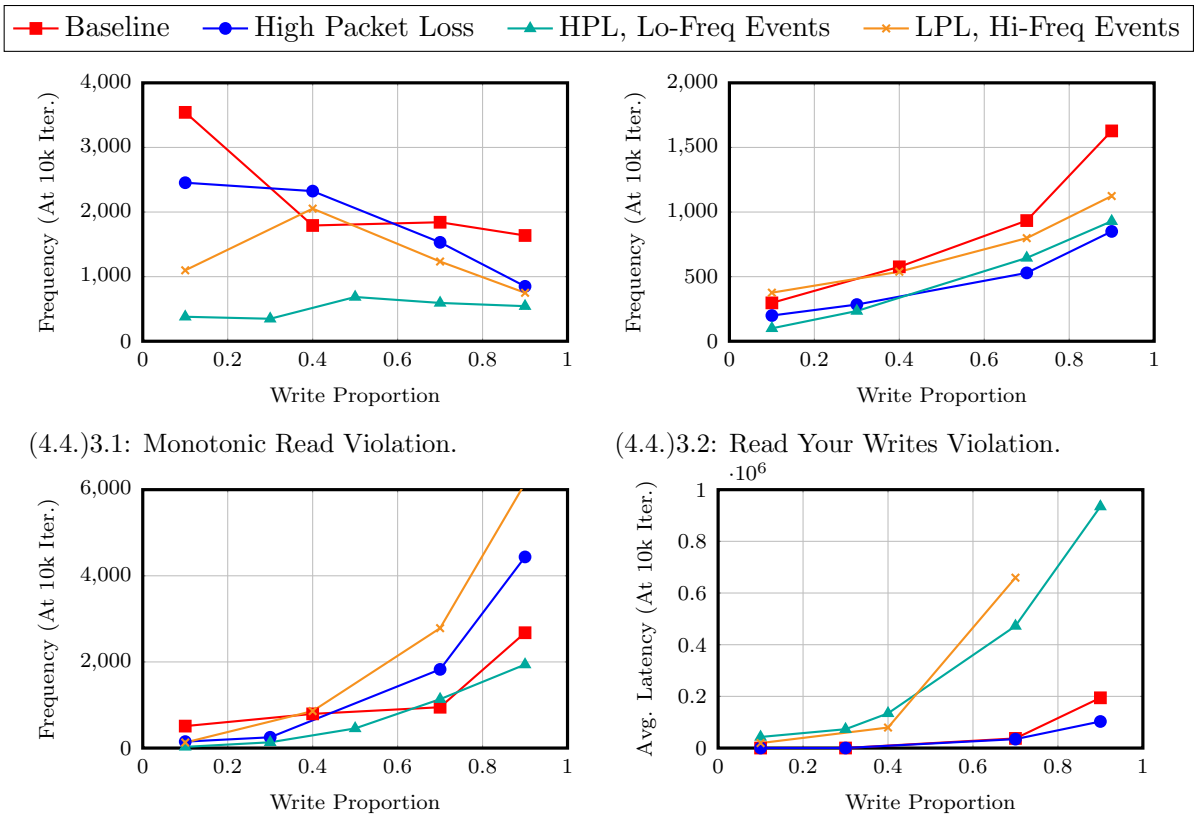
4.4.2.2: Read Your Writes Violation.

4.4.2.3: Monotonic Write Violation.

4.4.2.4: t-visibility.

Figure 4.4.2 Consistency Metrics Under Different Scalability
Read Prop. = 60%, Avg. Interval = 60 ms, Avg. Latency = 40 ms, Packet Loss Rate = 0.388%

In Figure 4.4.2 the green line (Large Cluster) shows the most violations for 3 out of 4 Metrics. Similarly, red line (Small Cluster) performs best for the same 3 metrics; blue line (Medium Cluster) performs second best. $Small > Medium > Large$ in terms of Cluster Size's contribution to the system consistency, if excluding the irregularity of the RYWC violation metric.



(4.4.)3.1: Monotonic Read Violation.

(4.4.)3.2: Read Your Writes Violation.

(4.4.)3.3: Monotonic Write Violation.

(4.4.)3.4: t-visibility.

Figure 4.4.3

	Packet Loss Rate	Avg. Event Interval (ms)	Avg. Latency (ms)	Node
Baseline	0.388%	60	40	8
High Packet Loss	1%	60	40	8
Hi Packet Loss, Lo-freq Events	1%	180	40	8
Lo Packet Loss, Hi-freq Events	0.388%	25	40	8

Subplot (4.4.)3.1 shows that, as Write Proportion increases, under the 4 settings, Monotonic Read violations decrease at 10,000 iterations/event count, while subplot 3.2 shows that RYW violations increase. Subplots 3.3 and 3.4 both indicate a poorer consistency when write proportion increases. All 4 subplots combined offer a complex pattern that is not identifiable without further discussion later.

5
Discussion

In the preceding discussion, it was assumed that the system implemented *Lamport's Timestamp*, theoretically ensuring Causal Consistency. During the experiments, components such as latency, packet loss rate (PLR), cluster size, and event frequency were controlled. Under the premise of causality consistency held, these settings, if leading to message loss or message out-of-order, would broadcast incorrect causal dependencies throughout the system. These causalities are deemed erroneous because they are not identical to the order of Version IDs the external simulation module generates. Consider when it is externally known that vid_4 is a Write to vid_3, not to vid_2 nor vid_5. Due to latency, message out-of-order establishes an incorrect causality chain. For example, if $\text{vid}_4 \rightarrow \text{vid}_2$ already exists and another incorrect causality $\text{vid}_2 \rightarrow \text{vid}_1$ is added, the transitivity of happen-before order implies that $\text{vid}_4 \rightarrow \text{vid}_1$ is also implied. Since the Causality Consistency depends on the entire version chain's history, such errors propagate and accumulate, as observed in the metrics. In Figure 4.4.1, 4.4.2, the 4 metrics increase as event count increases, validating this conclusion.

5.1 Results Analysis

5.1.1 Read/Write Proportion

Overall, a higher write proportion leads to poorer level of consistency. This is evident in Figure 4.4.3, where the RYWC violation, MRC violation, and t-visibility (Subplot 3.2, 3.3, 3.4) metrics increase as the write proportion increases. However, Subplot 4.4.3.1 demonstrates that as the write proportion increases, Monotonic Read violations decrease. A similar phenomenon is observed in Subplot 4.4.1.1, the blue line representing write-heavy simulation remains the lowest in the Monotonic Read violation metric.

One possible explanation is that, under latency, consecutive increasing Version IDs arriving at different Nodes are very likely to be processed out of order. In read-heavy, write-sparse scenarios, many out-of-order events are recorded therefore. In contrast, write-heavy scenarios offer chances for a very recent write to fix that out-of-order, and satisfy the MRC model during the measuring time window. If the majority of orders are wrong, any fix would lead to better a result. As long as the two most recent Version IDs in the version chain have a valid order, the MRC model is satisfied. Thus, the MRC violation metric fails to reflect the impact of the read/write proportion on distributed system consistency effectively.

5.1.2 Latency

Excluding the influence of the MRC violation metric, Subplots 4.4.1.2, 4.4.1.3, and 4.4.1.4 show that higher latency leads to more consistency violations. Latency increases the time required for the system to reach Eventual Consistency (EC) from a most recent write. Due to transmission delays, numerous messages remain unavailable within `Exchange`, while simulation events continue to occur, accumulating false causal dependencies.

For Subplot 4.4.1.3, one possible explanation is that the Monotonic Write metric records the frequency of inconsistencies throughout the past version chain. Write-heavy systems are inherently disadvantaged because version chains only record write operations. In high-latency

environments, a node is more likely to temporarily maintain partially correct causality as a late messages are still unavailable, stored in `Exchange`, while local read/write operations are still occuring and are immediately recorded with the order of external invocation, the correct causality. In contrast, in low-latency environments, local read/write operations are more likely to be interpolated with numerous incoming messages that are out-order-order due to randomized latency, resulting in higher overall MWC violation metrics.

5.1.3 Scalability

Figure 4.4.2 intuitively shows that larger cluster results in higher inconsistency. The irregularity of the RYWC violation metric (Subplot 4.4.2.2) is because this metric examines a Node's current Version ID against the Version ID of the last local write event. For large clusters (size = 200), local write events are sparsely distributed across all nodes, making it more likely for a Node receiving a *sync message* to satisfy the RYWC model. This is because the last recorded local write event is significantly older on average compared to smaller clusters, where local write events are denser. With sparser writes, the probability to trigger violations of RYWC is lower, but nothing is implied about the overall consistency level.

5.1.4 Message Loss

In Figure 4.4.3, comparing the red lines (Baseline) with the blue lines (High PLR) across all 4 subplots, systems with high PLR perform roughly better than the Baseline under various write proportions. High PLR causes more sync messages to be lost, resulting in more incomplete version chains. However, the metrics that are expected to reflect significantly higher inconsistency contrast with that. The results indicate that the (1) PLR difference of 1% vs. 0.388% do not lead to a significant effect on consistency for the system, therefore unmeasurable yet, or (2) the metrics applied and their designs fail to capture the impact of PLR on consistency.

5.1.5 Event Frequency

In Figure 4.4.3, comparing the red lines (Baseline, Avg. Interval = 60ms) with the orange lines (Hi-Freq, Avg. Interval = 25ms) across all four 4 subplots, Hi-Freq shows significantly higher inconsistency than Baseline in MWC violations, RYWC violations, and t-visibility. The MWC metric measures the degree of errors across the entire version chain, while t-visibility computes the total time required for a complete data synchronization. Both metrics directly reflect causal consistency in some way. If the latency t_M of message $M(\text{vid}_n)$ is greater than the event time interval t_v between vid_n and vid_{n+1}, causal inconsistency inevitably occurs because the causal dependency $\text{vid}_n \rightarrow \text{vid}_{n+1}$ is lost. Under the Hi-Freq setting (Avg. Interval = 25ms, Avg. Latency = 40ms), this is exactly the case, leading to high inconsistency metrics, especially in MWC violations and t-visibility.

5.2 Conclusion

This project developed and evaluated a simulation of distributed systems leveraging Lamport's Logical Clock algorithm to explore the dynamics of consistency across nodes under varying conditions. The simulation features a peer-to-peer system with autonomous nodes executing read/write operations, maintaining version chains for causal consistency. By using metrics such as Monotonic Read Violations, Read Your Writes Violations, Monotonic Write Violations and t-visibility, this study quantified the effects of latency, packet loss, event frequency, and scalability on system performance.

The results demonstrated several key insights:

Operational Dynamics: A higher proportion of write operations introduced greater inconsistency across most metrics, except for MRC violations, where write-heavy scenarios improved results due to their ability to override out-of-order reads.

Impact of Latency: The increase in latency significantly affected consistency metrics, highlighting its central role in synchronization delays and the propagation of erroneous causal dependencies.

Scalability Challenges: Larger cluster sizes exacerbated inconsistencies due to the sparse distribution of local writes and amplified synchronization cost, underlining the trade-offs inherent in scaling distributed systems.

Packet Loss and Event Frequency: Although packet loss exhibited a limited impact on consistency in this study, high event frequencies drastically increased violations due to overlapping operations outpacing synchronization mechanisms.

The innovation of this study lies in its meticulous implementation of logical clocks, metric design, and experimental framework, which collectively provide a platform to analyze the consistency of distributed systems. These findings to a certain degree validate the theoretical foundations of causal consistency and offer insights for developing and optimizing distributed systems.

5.3 Future Work

Alternative Logical Clock: Test whether a vector could enhance the accuracy of causal ordering, particularly in scenarios with high latency or frequent concurrent events.

Fault-Tolerant Enhancements: Extending the simulation to include Byzantine failures and their impact on consistency would provide a more comprehensive understanding of the behavior of the real-world system.

Dynamic Topologies: Simulating dynamic network topologies with changing cluster sizes (add, remove) and node responsibilities could better reflect the realities of distributed system deployments.

Real-World Implementation: Translating the simulation framework into a prototype on cloud platforms such as AWS or GCP would enable validation under practical workloads and infrastructure constraints.

These directions aim to bridge the gap between theoretical simulations and real-world applications.

Appendix A
Source Code

The source code of the simulation system is available at:

`https://github.com/zy035c/sproj-bard`

Bibliography

[1] Andrew S. and Van Steen Tanenbaum Maarten, *Distributed Systems*, 4th, Pearson, Harlow, England, 2023.

[2] Leslie Lamport, *Paxos Made Simple*, ACM SIGACT News **32** (2001), no. 4, 51–58, DOI 10.1145/568425.568433.

[3] Diego and Ousterhout Ongaro John, *In Search of an Understandable Consensus Algorithm* (2014).

[4] Allyx and Mosbah Fontaine Mohamed and Tounsi, *A Fault-Tolerant Handshake Algorithm for Local Computations*, 2016 30th International Conference on Advanced Information Networking and Applications Workshops (WAINA), 2016, pp. 663–668, DOI 10.1109/WAINA.2016.159.

[5] Leslie Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*, Communications of the ACM **21** (1978), no. 7, 558–565.

[6] ———, *How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs*, IEEE Transactions on Computers **28** (1979), no. 9, 690–691.

[7] Seth and Lynch Gilbert Nancy, *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*, ACM SIGACT News **33** (2002), no. 2, 51–59.

[8] Kourosh Gharachorloo, *Memory Consistency Models for Shared-Memory Multiprocessors*, Stanford University, 1995.

[9] David and Kuhlenkamp Bermbach Jörn, *Consistency in Distributed Storage Systems: An Overview of Models, Metrics, and Measurement Approaches* (2013).

[10] *Highly Available Transactions: Virtues and Limitations* (2014).

[11] Carson Banks Nelson, *Discrete-Event System Simulation*, Prentice Hall, Inc, 2000.